

システムプログラミングⅡ 令和7年度 前期末試験

(2025.07.31 重村 哲至) IE4 番 氏名 模範解答

付録4にヒントがあるので適宜参照すること。

1 語句に関する問題

空欄(1)～(15)に最適な言葉を語群から記号で答えなさい。(1点×15問=15点)

(1)システムコールを用いて子プロセスを作り、子プロセスが(2)システムコールを用いて新しいプログラムを実行することで、新しいプロセスで新しいプログラムを実行することができる。この方式は(3)方式と呼ばれる。一方で一つのシステムコールだけ用いる方式は(4)方式と呼ばれる。

(1)システムコールが返す値は、子プロセスでは0、親プロセスでは子プロセスの(5)である。(2)システムコールが戻るのは(6)が発生した時だけであるので、(2)システムコールを呼び出した次の行から(6)処理を開始してよい。(2)システムコールの引数は第1引数から順に、実行するファイルの(7)、(8)配列、(9)配列である。

(10)は、(2)システムコールの実行前に、(11)をファイルに接続しておくことで実現される。実行されるプログラムではなく(12)が(10)機能を実現している。

日付や時刻の表示に使用する言語は(13)環境変数によって制御される。日本語で表示したい場合は(13)環境変数に(14)のような値を設定する。(15)環境変数には、タイムゾーンを設定することができる。

語群：(あ) execve, (い) fork, (う) fork-exec, (え) ja_JP.UTF-8, (お) LC_TIME, (か) PID(プロセス番号), (き) spawn, (く) TZ, (け) エラー, (こ) 環境変数, (さ) コマンド行引数, (し) シェル, (す) パス, (せ) 標準入出力, (そ) リダイレクト

(1)	(い)	(2)	(あ)	(3)	(う)	(4)	(き)
(5)	(か)	(6)	(け)	(7)	(す)	(8)	(さ)
(9)	(こ)	(10)	(そ)	(11)	(せ)	(12)	(し)
(13)	(お)	(14)	(え)	(15)	(く)		

2 Cプログラムの実行結果

1. 付録1のp1の実行結果を答えなさい。(5点)

Y

2. 付録1のp2の実行結果を答えなさい。(5点)

NULL

3. 付録1のp3を次のように実行したときに作成されるテキストファイルの名前と内容を答えなさい。ファイルの内容は文字列で答えること。(5点)

% ./p3 a.txt b.txt

ファイル名	a.txt
ファイル内容	b.txt

4. 付録1のp4の実行結果を答えなさい。(5点)

abc

5. 付録1のp5の実行結果を答えなさい。(5点)

abc

123

6. 付録1のp6の実行結果を答えなさい。(5点)

abc

abc

def

7. 付録1のp7の実行結果を答えなさい。(5点)

123

def

システムプログラミングⅡ 令和7年度 前期末試験

(2025.07.31 重村 哲至)

IE4

番 氏名

模範解答

3 環境変数进行操作する C プログラム

付録2のプログラムが使用できます。

1. 以下の操作をしたときの出力を答えなさい。なお、何も出力されない場合は「出力なし」、エラーが発生する場合は「エラー」と答えなさい。(4点×6問=24点)

(a) myprintenv 単独で実行した場合。

```
% export A=B
% export B=A
% ./myprintenv A
```

B

(b) myenv と組み合わせた場合。

```
% export A=B
% ./myenv A=C B=D ./myprintenv A
```

C

(c) myenv と組み合わせた場合。

```
% export A=B
% ./myenv A=C B=D ./myprintenv A
% ./myprintenv A
```

C

B

(d) zeroenv と組み合わせた場合。

```
% ./zeroenv B=C ./myprintenv
```

エラー

(e) zeroenv と組み合わせた場合。

```
% ./zeroenv ./myprintenv
```

出力なし

(f) zeroenv, myenv と組み合わせた場合。

```
% ./zeroenv ./myenv A=C B=D ./myprintenv
```

A=C

B=D

2. 前の問題でエラーが発生するものが一つだけあります。なぜエラーが発生するのか説明しなさい。(4点)

(d) では、zeroenv の引数に「B=C ./myprinenv」を与えている。zeroenv は引数をコマンドだと解釈するので、「B=C」をコマンド名だと思って execve システムコールに渡してしまう。「B=C」というコマンドはないので execve システムコールが実行ファイルを見つけることができずエラーになる。zeroenv プログラムは「B=C: No such file or directory」と表示して終了する。

4 シェル

1. 必ずシェルの内部コマンドにするべきものに「○」、外部コマンドでも実現可能なものに「×」を書きなさい。(3点×6問=18点)

cd	○	pwd	×
export	○	unset	○
printenv	×	env	×

2. 内部コマンドにしなければならないコマンドに共通の特徴を説明しなさい。(4点)

シェルプロセス自身の状態を変化させるもの。例えば cd コマンドはシェルプロセスのカレントディレクトリを変更する。外部コマンドは別のプロセスで実行されるので、シェルプロセスのカレントディレクトリを変更することはできない。

システムプログラミングⅡ 令和7年度 前期末試験

(2025.07.31 重村 哲至)

IE4

番 氏名

模範解答

付録1：テストプログラム

```
// p1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(int argc, char *argv[]) {
    char *v;
    setenv("X", "Y", 0);
    setenv("X", "Z", 0);
    if((v=getenv("X"))!=NULL) {
        printf("%s\n",v);
    } else {
        printf("NULL\n");
    }
    return 0;
}
```

```
// p2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(int argc, char *argv[]) {
    char *v;
    setenv("X", "Y", 0);
    setenv("X", "Z", 0);
    if((v=getenv("Z"))!=NULL) {
        printf("%s\n",v);
    } else {
        printf("NULL\n");
    }
    return 0;
}
```

```
// p3.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main(int argc, char *argv[]) {
    close(1);
    int fd = open(argv[1],
                  O_WRONLY|O_TRUNC|O_CREAT, 0644);
    write(fd, argv[2], strlen(argv[2]));
    write(fd, "\n", 1);
    return 0;
}
```

```
// p4.c
// #includeの掲載省略
int main(int argc, char *argv[]) {
    printf("abc\n");
    exit(0);
    printf("def\n");
    return 0;
}
```

```
// p5.c
// #includeの掲載省略
int main(int argc, char *argv[]) {
    printf("abc\n");
    execlp("echo", "echo", "123", NULL);
    printf("def\n");
    return 0;
}
```

```
// p6.c
// #includeの掲載省略
int main(int argc, char *argv[]) {
    int pid = fork();
    printf("abc\n");
    if (pid==0) {
        exit(0);
    } else {
        int stat;
        wait(&stat);
    }
    printf("def\n");
    return 0;
}
```

```
// p7.c
// #includeの掲載省略
int main(int argc, char *argv[]) {
    if (fork()!=0) {
        int stat;
        wait(&stat);
        printf("def\n");
    } else {
        execlp("echo", "echo", "123", NULL);
        printf("456\n");
        exit(0);
    }
    return 0;
}
```

システムプログラミングⅡ 令和7年度 前期末試験

(2025.07.31 重村 哲至)

IE4

番 氏名

模範解答

付録2：環境変数に関するプログラム

```
// myenv.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
extern char **environ;
int main(int argc, char *argv[]) {
    for (int i=1; argv[i]!=NULL; i++) {
        if (putenv(argv[i])!=0) {
            execvp(argv[i], &argv[i]);
            perror(argv[i]);
            return 1;
        }
    }
    for (int i=0; environ[i]!=NULL; i++) {
        printf("%s\n", environ[i]);
    }
    return 0;
}
```

```
// myprintenv.c
#include <stdio.h>
#include <stdlib.h>
extern char **environ;
int main(int argc, char *argv[]) {
    if (argc==1) {
        for (int i=0; environ[i]!=NULL; i++) {
            printf("%s\n", environ[i]);
        }
    } else {
        char *env = getenv(argv[1]);
        if (env==NULL) return 1;
        printf("%s\n", env);
    }
    return 0;
}
```

```
// zeroenv.c
#include <stdio.h>
#include <unistd.h>
char *env[] = { NULL }; // 空の環境変数配列
int main(int argc, char *argv[]) {
    execve(argv[1], &argv[1], env);
    perror(argv[1]);
    return 1;
}
```

付録4：ヒント

C 言語の関数・変数やシステムコール

```
// ファイル関係
int open(char *path, int oflag, ...);
oflag:
    O_RDONLY (読み込み)
    O_WRONLY (書き込み)
    O_CREAT (ファイル作成)
    O_TRUNC (ファイル切詰め)
int close(int fd);
int read(int fd, void *buf, int nbyte);
int write(int fd, void *buf, int nbyte);
// 環境変数に関する関数・変数
extern char **environ;
char *getenv(char *name);
int setenv(char *name, char *val, int overwrite);
int putenv(char *string);
int unsetenv(char *name);
// プログラム実行関係
int execve(char *path,
            char *argv[], char *envp[]);
int execvp(char *file,
            char *argv[], char *envp[]);
int execl(char *path, char *argv0,
            *argv1, ... , argvn, NULL);
int execlp(char *file, char *argv0,
            *argv1, ... , argvn, NULL);
// プロセス生成・終了関係
int fork(void);
void exit(int status);
pid_t wait(int *status);
```